A Method for Automatically Extracting ABAC Attributes from Natural Language Policy Statements

Duc-Hieu Nguyen[1*], Nhat-Anh Phung[2], Manh-Quyen Nguyen[2], Dinh-Tan Nguyen[2], and Huy-Khanh Anh Tran[2]

[1] The Academy of Cryptography Techniques, Hanoi 12510, Vietnam
ndhieu@actvn.edu.vn
[2] The Academy of Cryptography Techniques, Hanoi 12510, Vietnam
AT210402@actvn.edu.vn, CT060236@actvn.edu.vn,
CT070248@actvn.edu.vn, CT080201@actvn.edu.vn

Abstract. In modern information systems, access control policies are often written in natural language in high-level requirements specification documents, business documents, or internal regulations. Attribute-based access control (ABAC) models enable authorization decisions based on attributes of subjects, objects, and environments, but manually extracting these attributes from natural language access control policy statements is time-consuming and error-prone. This paper presents a method for automatically extracting ABAC attributes from English-language policy statements; the main contribution is the extraction of environmental attributes using a three-layer algorithm. In our approach, we leverage existing natural language processing techniques and the DBSCAN algorithm. Additionally, we use the F1-score metric to evaluate the performance of the proposed pipeline. Experimental results on the iTrust real-world and VACT synthetic datasets show that our pipeline achieved overall F1-scores of 0.98 and 0.85, respectively, and achieved F1-scores of 0.88 and 0.98, respectively, when extracting environmental attributes.

Keywords: ABAC, Attribute Extraction, Natural Language Policy Statements, Natural Language Processing, Environmental Attributes, DBSCAN.

1 Introduction

In the context of digital transformation, information systems in organizations and businesses need to handle a large number of access control rules. These access control rules are often not specified as policies that computers can understand and execute; they are recorded in high-level requirements specification documents or business description documents as natural language policy statements (NLPSs). On the other hand, access control policies (ACPs) are often studied and represented based on access control models; among the popular access control models, attribute-based access control (ABAC) [9, 10] is a powerful model in meeting the detailed and specific security requirements of modern applications [11]. ABAC can make dynamic access control decisions by transforming attribute values without altering the relationships

among subject, object, and environment [13], the three core concepts of the original ABAC model [15]. Therefore, extracting ABAC attributes related to the subject, object, and environment from NLPSs is necessary. This extraction process is often done manually, leading to time-consuming and error-prone processes.

In the literature, there are two notable gaps regarding the extraction of ABAC components from NLPSs. Automated methods for extracting ABAC components from natural language documents have been proposed in prior studies [1, 2, 4, 5]. However, these studies do not address the extraction of environmental attributes—the first gap. The second gap is the lack of datasets (i.e., NLPSs) that can be marked with environmental labels. The datasets used in studies on the extraction of ABAC attributes, including iTrust [6], CyberChair [17], IBM Course Management App [18], and Collected ACP [19], contain little or no environmental information.

This paper presents a pipeline for the simultaneous extraction of ABAC attributes from NLPSs, including subject, object, and environment attributes. The main contribution of the paper is to address the two gaps mentioned above, namely: (1) automatic extraction of environmental attributes from NLPSs, leveraging existing natural language processing techniques; (2) providing a synthetic dataset of 100 NLPSs (VACT) describing the access control rules currently in effect at the Academy of Cryptography Techniques in Hanoi, Vietnam.

The rest of this paper is organized as follows. Section 2 presents the studies related to our work. In Section 3, we present our pipeline in detail. Section 4 describes the experimental implementation and evaluation. Section 5 discusses the validity, efficiency, novelty, and limitations of our pipeline before concluding in Section 6.

2 Related Studies

Regarding the automatic extraction of ABAC components from natural language documents, a practical framework has been proposed to extract the delegate attributes of hierarchical ABAC systems using natural language processing and machine learning techniques [1]. However, this study does not extract the environmental attributes. Alohaly et al. [2] designed a framework to extract ABAC constraint expressions from NLPSs; however, it does not extract environment attributes. Several transformer-based models have been evaluated to automatically extract key components of natural language ACPs [8]. However, this study also does not extract the environmental attributes.

We found four studies on the automatic extraction of ACPs from natural language documents [3, 5, 7, 12]. Narouei et al. [3] used a semantic role labeling technique to identify ACP elements; however, this study is not specifically designed for ABAC. Shan et al. [5] proposed a deep learning-based method to automatically generate ABAC policies. Zhang et al. [7] introduced a lightweight and efficient framework that combines named entity recognition with language models to extract fine-grained ACPs. Slankas and Williams [12] proposed a machine learning-based process for analyzing existing natural language documents to extract subjects, actions, and resources related to ACPs. However, these studies do not extract environmental attrib-

utes. The following Table 1 summarizes the scope of extraction for the aforementioned related studies.

Table 1. The scope of extraction for the related studies.

Related studies	Scope of extraction	Extraction of environment attributes
[1], [2]	Subject attributes, object attributes, and ABAC constraints	None
[3]	Subjects, objects, and actions	None
[5]	Subject attributes, object attributes, and actions	None
[7]	Fine-grained ACPs	None
[12]	Subjects, objects, and actions	None
This paper	Subject attributes, object attributes, environment attributes, and actions	Available

3 Extracting ABAC Attributes from NLPSs

3.1 Proposed Pipeline

This subsection provides an overview of our pipeline for extracting ABAC attributes from NLPSs. The pipeline's input is NLPSs, and its output is a JSON containing standardized ABAC policy constructs. The steps in our pipeline, their tasks, and the output of each step are summarized in the following Table 2.

Table 2. The steps in our pipeline.

Steps	Tasks	Outputs
1	Extract environmental attributes first	Valid environment tokens
2	Extract actions, subject attributes, and object attributes	Valid action-subject-object tokens
3	Overlap pruning	Pruned action-subject-object-environment attributes
4	Human-in-the-loop verification	Gold-standard dataset
5	Normalization, type inference, and clustering	Standardized ABAC policy constructs (JSON)

The proposed pipeline consists of three stages: Stage 1 (see Fig. 1), Stage 2 (see Fig. 2), and Stage 3 (see Fig. 3). The following describes the three stages in detail.

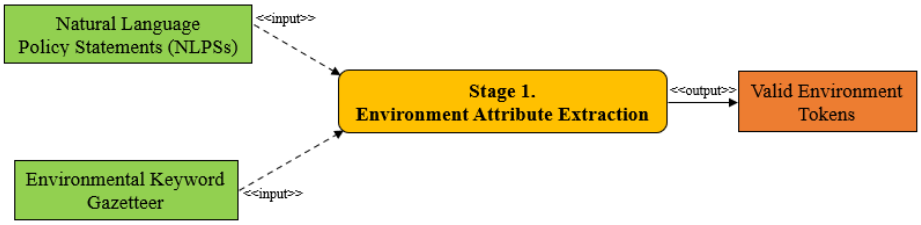


Fig. 1. Stage 1—Environment Attribute Extraction.

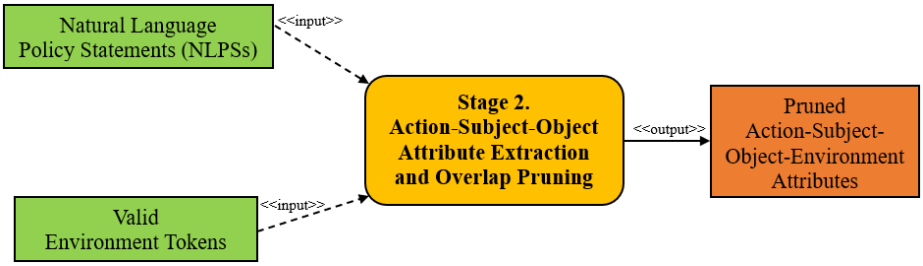


Fig. 2. Stage 2—Action-Subject-Object Attribute Extraction and Overlap Pruning.

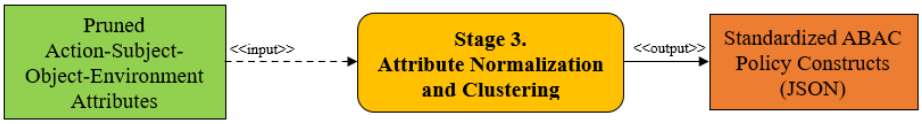


Fig. 3. Stage 3—Attribute Normalization and Clustering.

Stage 1—Environment Attribute Extraction. The input for this stage is NLPSs and an environmental keyword gazetteer. In our approach, the *env_extractor.py* file would identify raw environment entities, and then the *relation_candidate.py* file would identify candidate environment attributes using dependency parsing. The output of this stage is valid environment tokens.

Stage 2—Action-Subject-Object Attribute Extraction and Overlap Pruning. The input for this stage is the NLPSs and the valid environment tokens (i.e., the output of Stage 1). In our approach, the *auto_extraction.py* file would be executed to automatically extract actions, subject attributes, object attributes, and relationship pairs. We leverage dependency patterns in [1], including *amod*, *compound*, *acl*, *prep*, and *nummod*. Actions would be mapped as follows: view→Read, modify→Update, delete→Delete. Then, the *interactive_verification.py* file would give an interface for human-in-the-loop verification, i.e., assist users in reviewing, editing, and removing overlapping attributes. The output of this stage is pruned subject attributes, object attributes, environment attributes, and actions. These results would be stored in a JSON file, which serves as the gold-standard dataset for calculating *precision*, *recall*, and *F1-score* metrics.

Stage 3—Attribute Normalization and Clustering. The input for this stage is the pruned subject attributes, object attributes, environment attributes, and actions (i.e., the output of Stage 2). We would read data from a JSON file containing the policy

dataset, perform valid attribute classification, assign hierarchical namespaces, make type inference, and employ the DBSCAN (density-based spatial clustering of applications with noise) algorithm [20] to cluster attributes with similar meanings (e.g., "business hours," "working hours," and "office hours" are grouped together in the *working_period* cluster).

The proposed pipeline's output is extracted ABAC attributes in JSON. To illustrate, an NLPS like "A senior nurse can view medical records during business hours within the hospital" would produce the following extraction results: the extracted subject is senior_nurse, the extracted action is Read, the extracted object is medical_records, the extracted environment attributes are business_hours and hospital.

3.2 Three-Layer Algorithm for Extracting Environmental Attributes

In our approach, extracting environment attributes is realized via the following three-layer algorithm.

Layer 1—Trigger Detection. We leverage the *spaCy* tool [14] and NER (named entity recognition) and dependency parsing techniques, along with an environmental keyword gazetteer, to detect candidate environmental phrases. The gazetteer contains trigger keyword groups, including *TEMPORAL_PREPS* (*during*, *before*, *after*, *between*), *SPATIAL_PREPS* (*within*, *at*, *in*), *NETWORK_DEVICE_TRIGGERS* (*via VPN*, *from intranet*, *registered device*), and conditional conjunctions (*if*, *when*, *only*, *unless*).

Layer 2—Disambiguation. We check candidate environmental phrases in the dependency tree and for noun-phrase containment. In more detail, if a candidate environmental phrase is within the subject or object's noun phrase containment, it will be excluded; conversely, it will be considered an environmental attribute (e.g., "hospital nurse" is within the subject's noun phrase containment since "hospital" modifies "nurse;" conversely, "within the hospital" is considered an environmental attribute).

Layer 3—Classification and Hierarchical Namespace Assignment. We categorize valid environment attributes into the following groups: *temporal*, *spatial_physical*, *spatial_network*, and *spatial_device*. We then assign hierarchical namespaces to ensure consistency in the extracted results (e.g., environment.time.working_period:business_hours, environment.location.physical: hospital).

4 Experimental Implementation and Evaluation

4.1 Experimental Settings

Our pipeline was applied to two datasets: iTrust and VACT. The **iTrust** real-world dataset comprises 53 NLPSs in the medical domain (see [6] for details). The **VACT** dataset in the education domain comprises 100 manually written NLPSs describing 100 access control rules currently in use at the Academy of Cryptography Techniques in Hanoi, Vietnam (see [16] for details). These two datasets are unstructured NLPSs

written in English. The experimental environment included: *Python 3.8+*, *spaCy 3.x (en_core_web_md)*, *scikit-learn for DBSCAN*, and *numpy for the k-distance graph*.

4.2 Experimental Results

We employ *precision*, *recall*, and *F1-score* metrics to evaluate the experimental performance. The *precision* metric measures the accuracy of correct extractions; the recall metric measures the coverage of extracted ABAC attributes (i.e., missing ABAC attributes not extracted); and the *F1-score* is the overall evaluation metric. These metrics are calculated using the following formulas (where TP stands for True Positive, FN stands for False Negative, FP stands for False Positive):

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{1}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{2}$$

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3}$$

The experimental results on the iTrust and VACT datasets are presented in Tables 3 and 4, respectively.

Table 3. Experimental results on the iTrust dataset.

ABAC attributes	Precision	Recall	F1-score
Subject	1.00	1.00	1.00
Action	0.94	1.00	0.97
Object	0.98	1.00	0.99
Environment	1.00	0.80	0.88
Overall	0.97	0.98	0.98

Table 4. Experimental results on the VACT dataset.

ABAC attributes	Precision	Recall	F1-score
Subject	0.90	0.90	0.90
Action	0.96	0.97	0.97
Object	0.49	0.49	0.49
Environment	0.97	1.00	0.98
Overall	0.84	0.85	0.85

4.3 Experimental Evaluation

Experimental results indicate that the pipeline efficiently handles the core components of ABAC, including subject, action, and environment. With the iTrust dataset, envi-

ronmental attribute extraction achieved an F1-score of only 0.88 because several NLPSs have complex conditional structures or nested prepositional phrases. With the VACT dataset, environmental attribute extraction achieved an F1-score of 0.98 (i.e., the extraction results do not miss any environmental attributes). However, object extraction achieved a very low F1-score of 0.49 due to the more complex NLPS structure in the VACT dataset compared to the iTrust dataset, leading to an overall F1-score of only 0.85 for the VACT dataset.

5 Discussion

Validity. Our pipeline has been successfully adopted to the two NLPS datasets from the two different domains. This partly demonstrates the validity of the proposed pipeline.

Efficiency. Overall, F1-score results in Tables 3 and 5 indicate that our pipeline is relatively efficient when extracting ABAC attributes from NLPSs. Compared to the work of Alohaly et al. [1], the subject and object attribute extraction performance of our pipeline is lower, as we did not design a dedicated pipeline for subject and object attribute extraction; instead, we extended it for environment attribute extraction.

Novelty. Our pipeline can simultaneously extract subject, object, and environmental attributes, as well as actions—a novelty of this paper compared with previous studies.

Limitations. Our pipeline depends on an environmental keyword gazetteer and struggles to handle NLPSs that contain nested prepositional phrases, complex conditional structures, or multi-layered compound noun phrases. Furthermore, the experimental part of this paper was implemented only on two small-scale NLPS datasets.

6 Conclusion

This paper presents a method for automatically extracting ABAC attributes from NLPSs. The main contribution of this paper is the extraction of environmental attributes using a three-layer algorithm comprising Trigger Detection-Disambiguation-Classification and Hierarchical Namespace Assignment. Our pipeline can simultaneously extract subject, object, and environmental attributes, as well as actions. Another contribution is the VACT dataset, a publicly available NLPS dataset for environmental labeling.

Future work will involve conducting experiments on large-scale NLPS datasets to further evaluate the applicability of our approach. Furthermore, integrating large language models into the pipeline to enhance efficiency is also a promising direction for future research.

References

1. Alohaly, M., Takabi, H., Blanco, E.: Automated extraction of attributes from natural language attribute-based access control (ABAC) policies. Cybersecurity 2(1), 2 (2019).
2. Alohaly, M., Takabi, H., Blanco, E.: Towards an automated extraction of abac constraints from natural language policies. In: IFIP international conference on ICT systems security and privacy protection, pp. 105-119. Springer International Publishing, Cham (2019).
3. Narouei, M., Takabi, H., Nielsen, R.: Automatic Extraction of Access Control Policies from Natural Language Documents. IEEE Transactions on Dependable and Secure Computing 17(3), 506-517 (2018).
4. Abdelgawad, M., Ray, I., Alqurashi, S., Venkatesha, V., Shirazi, H.: Synthesizing and analyzing attribute-based access control model generated from natural language policy statements. In: Proceedings of the 28th ACM Symposium on Access Control Models and Technologies, pp. 91-98 (2023).
5. Shan, F., Wang, Z., Liu, M., Zhang, M.: Automatic Generation of Attribute-Based Access Control Policies from Natural Language Documents. Computers, Materials & Continua 80(3), (2024).
6. GitHub, https://github.com/nhatanh19-git/ABAC/blob/main/NLACP-AttributeExtractor-master/ACP_dataset/iTrust_ACP.txt, last accessed 2026/07/08.
7. Zhang, Y., Xu, Z., Huo, D., Guo, X., Zhou, Q., Wang, Y.: LitroACP: A Lightweight and Robust Framework for Extracting Access Control Policies from Specifications. In: International Conference on Advanced Information Systems Engineering, pp. 75-90. Springer Nature Switzerland, Cham (2025).
8. Petrillo, L., Martinelli, F., Santone, A., Mercaldo, F.: An Explainable Method for Automatic Extraction of Natural Language Access Control Policy Key Components. Applied Sciences 15(22), 11854 (2025).
9. Yuan, E., Tong, J.: Attributed based access control (ABAC) for web services. In: IEEE International Conference on Web Services (ICWS'05). IEEE (2005).
10. Hu, V. C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., Scarfone, K.: Guide to attribute based access control (ABAC) definition and considerations. NIST special publication 800(162), 1-54 (2014).
11. Nguyen, D. H.: Formalizing and Conformance Testing ABAC Policies in Process Logs via an ABAC Metamodel. In: 2025 17th International Conference on Knowledge and System Engineering (KSE), pp. 1-7. IEEE (2025).
12. Slankas, J., Williams, L.: Access control policy extraction from unconstrained natural language text. In: 2013 International Conference on Social Computing, pp. 435-440. IEEE (2013).
13. Nguyen, D. H., Sei, Y., Tahara, Y., Ohsuga, A.: A model-based approach for designing and validating ABAC policies. In: Roger, L. (ed.) SCI 1193, Software Engineering and Management: Theory and Applications, vol. 17, pp. 19-36. Springer Nature Switzerland, Cham (2025).
14. Honnibal, M., Montani, I., Van Landeghem, S., Boyd, A.: spaCy: Industrial-strength natural language processing in Python. (2020).
15. Nguyen, D. H., Sei, Y., Tahara, Y., Ohsuga, A.: Model-driven approach enabling formalization and conformance testing of attribute-based access control policies for business processes. Journal of Computer Security, 0926227X261421496 (2025).
16. GitHub, https://github.com/nhatanh19-git/ABAC/blob/main/NLACP-AttributeExtractor-master/ACP_dataset/VACT_final.txt, last accessed 2026/07/08.

17. Van De Stadt, R.: Cyberchair: A web-based groupware application to facilitate the paper reviewing process. arXiv preprint arXiv:1206.1833 (2012).
18. Course registration requirements, https://khanhn.wordpress.com/wp-content/uploads/2016/08/vidu-ibm.pdf, last accessed 2026/07/08.
19. Xiao, X., Paradkar, A., Thummalapenta, S., Xie, T.: Automated extraction of security policies from natural-language software documents. In: Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering, pp. 1-11 (2012).
20. Khan, K., Rehman, S. U., Aziz, K., Fong, S., Sarasvady, S.: DBSCAN: Past, present and future. In: The fifth international conference on the applications of digital information and web technologies (ICADIWT 2014), pp. 232-238. IEEE (2014).